

INSTITUIÇÃO DE ENSINO

CURSO DE TECNOLOGIA EM SISTEMAS EMBARCADOS

**ESTAÇÃO METEOROLÓGICA IOT COM ESP32, MQTT
E NODE-RED**

Projeto de monitoramento ambiental com aquisição de dados, publicação em
nuvem e visualização em dashboard

MIGUEL SANTANA DA COSTA

MOGI DAS CRUZES

2026

ESTAÇÃO METEOROLÓGICA IOT COM ESP32, MQTT E NODE-RED

Miguel Santana Da Costa

Projeto apresentado ao curso de Tecnologia em Sistemas Embarcados, como requisito parcial para avaliação da disciplina de projetos integradores, com foco no desenvolvimento de uma estação meteorológica IoT baseada em ESP32, MicroPython, MQTT e Node-RED.

Orientador: Alessandro Aparecido da Silva Horas

Mogi das Cruzes

2026

Resumo

Este trabalho apresenta o desenvolvimento de uma estação meteorológica IoT baseada em ESP32, implementada em MicroPython e integrada a sensores ambientais, display OLED, botões físicos, broker MQTT em nuvem e dashboard em Node-RED. A proposta do projeto foi construir uma solução completa de monitoramento ambiental capaz de coletar dados locais, tratá-los no firmware, publicá-los de forma segura para um broker remoto e exibi-los em uma interface de supervisão em tempo real. O sistema foi estruturado com programação orientada a objetos, o que permitiu organizar o firmware em blocos independentes e reutilizáveis, facilitando leitura, manutenção, teste e expansão. Além disso, a implementação foi validada em ambiente simulado com Wokwi, o que possibilitou o desenvolvimento incremental e a verificação da lógica antes da implementação em hardware real.

Contents

1	Introdução	5
2	Objetivos do Projeto	5
3	Requisitos do Sistema	6
3.1	Requisitos Funcionais	6
3.2	Requisitos Não Funcionais	7
4	Arquitetura Geral do Sistema	8
4.1	Camada de Aquisição	8
4.2	Camada de Processamento	8
4.3	Camada de Comunicação	8
4.4	Camada de Consumo	9
5	Componentes de Hardware	9
5.1	ESP32	9
5.2	Sensor DHT22	9
5.3	Sensor LDR	9
5.4	Sensor MQ2	9
5.5	Display OLED SSD1306	10
5.6	Botões	10
6	Estrutura do Firmware em MicroPython	10
6.1	Configuração do Hardware	10
6.2	Abstração dos Sensores	11
6.3	Leitura do DHT22	11
6.4	Leituras Analógicas	12
6.5	Cálculos e Classificações	12
6.6	Coleta e Formatação dos Dados	12
6.7	Gerenciamento MQTT	13
6.8	Controle do Display	13
7	Fluxo de Dados	13

8	Configuração de MQTT	14
9	Node-RED e Docker	14
10	Simulação em Wokwi	14
11	Análise dos Requisitos	15
11.1	Descrição Detalhada dos Requisitos Funcionais	15
11.2	Descrição Detalhada dos Requisitos Não Funcionais	15
12	Conclusão	15

1 Introdução

A Internet das Coisas (IoT) tem ampliado significativamente a forma como sistemas físicos são monitorados e controlados. Em vez de depender exclusivamente de observação manual, dispositivos embarcados podem medir variáveis do ambiente, enviar dados para a nuvem e disponibilizar informações em tempo real para análise e tomada de decisão. Esse contexto é especialmente útil em aplicações ambientais, agrícolas, residenciais e industriais, nas quais temperatura, umidade, luminosidade e presença de gases influenciam diretamente o comportamento do sistema monitorado.

Dentro dessa perspectiva, o projeto desenvolvido consistiu na criação de uma estação meteorológica IoT baseada em ESP32, com suporte a sensores de temperatura e umidade, luminosidade, gás e elementos de interface local. O ESP32 foi escolhido por reunir conectividade Wi-Fi, ampla disponibilidade, custo acessível e compatibilidade com MicroPython. A solução proposta não se limitou à aquisição de dados: o firmware também processa as informações, classifica estados do ambiente, exibe resultados localmente e publica os valores em MQTT para posterior consumo pelo Node-RED.

Outro ponto importante do projeto foi a organização da lógica em classes. Essa decisão foi tomada para evitar um código monolítico e difícil de manter. Ao separar responsabilidades entre configuração de hardware, leitura de sensores, cálculo de estados, coleta de dados, envio MQTT e controle do display, tornou-se possível estruturar o sistema de forma mais clara e evolutiva. Dessa forma, o projeto não apenas cumpre sua função técnica, mas também demonstra uma abordagem de engenharia de software aplicada a sistemas embarcados.

2 Objetivos do Projeto

O objetivo geral foi desenvolver uma estação meteorológica IoT funcional, documentada e modular, capaz de operar em simulação e em hardware real. A intenção foi construir uma solução que unisse aquisição de dados, processamento embarcado, comunicação em rede e visualização dos resultados em dashboard.

Como objetivos específicos, o projeto buscou:

- captar dados ambientais por meio de sensores físicos;
- processar os dados localmente no ESP32;

- representar os valores em forma de estados e alertas;
- exibir informações em display OLED;
- permitir navegação entre telas por botões físicos;
- publicar os dados em um broker MQTT na nuvem;
- estruturar o firmware com orientação a objetos;
- possibilitar o consumo das mensagens em Node-RED;
- documentar o sistema de forma clara e reprodutível;
- validar o comportamento em simulação antes do uso em hardware real.

Esses objetivos ajudam a evidenciar que o projeto não foi apenas uma leitura de sensores, mas sim uma arquitetura completa de monitoramento com integração entre dispositivo embarcado e interface de supervisão.

3 Requisitos do Sistema

3.1 Requisitos Funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer. No caso desta estação meteorológica, o sistema precisa cumprir as seguintes funções:

- ler temperatura e umidade por meio do sensor DHT22;
- ler luminosidade por meio do sensor LDR;
- ler nível de gás por meio do sensor MQ2;
- exibir valores e estados no display OLED;
- alternar entre telas com botões físicos;
- conectar-se a uma rede Wi-Fi;
- autenticar-se em um broker MQTT remoto;
- publicar os dados em formato JSON;

- enviar mensagens periodicamente;
- manter reconexão automática em caso de falha de comunicação;
- fornecer uma interface de visualização em Node-RED;
- classificar o ambiente com base nos valores lidos;
- gerar alertas quando os parâmetros ultrapassarem limites definidos;
- registrar um timestamp para cada amostragem;
- operar em ambiente simulado e, com pequenas adaptações, em hardware real.

3.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem a qualidade e as restrições da solução. Neste projeto, eles foram definidos da seguinte forma:

- o código deve ser modular e legível;
- o firmware deve ser compatível com MicroPython;
- o sistema deve ser executável em ESP32;
- o uso de memória e processamento deve ser compatível com um microcontrolador;
- a comunicação MQTT deve utilizar conexão segura quando possível;
- a estrutura deve permitir expansão futura para novos sensores e telas;
- o sistema deve suportar manutenção simples;
- a documentação deve ser clara e reutilizável;
- a simulação deve reproduzir a lógica do sistema real;
- a interface de usuário deve ser compreensível e funcional;
- a solução deve ser escalável em termos de software;
- o projeto deve permitir integração com ferramentas de automação e monitoramento.

Esses requisitos foram ampliados para mostrar que o sistema não se limita à funcionalidade mínima, mas busca também organização, confiabilidade e extensibilidade.

4 Arquitetura Geral do Sistema

A arquitetura do sistema foi planejada em camadas para deixar o fluxo de funcionamento mais claro e para separar adequadamente as responsabilidades entre hardware, firmware e software de supervisão. Essa separação também facilita a substituição de componentes futuros sem reescrever todo o projeto.

4.1 Camada de Aquisição

A camada de aquisição corresponde ao conjunto de sensores e periféricos conectados ao ESP32. Nela estão o DHT22, o LDR, o MQ2, o display OLED e os botões físicos. Cada um desses elementos representa uma entrada ou saída específica do sistema.

O DHT22 fornece temperatura e umidade, variáveis fundamentais para um monitoramento meteorológico básico. O LDR mede a luminosidade ambiente, enquanto o MQ2 permite uma estimativa do comportamento do ar em relação à presença de gases combustíveis. O display OLED apresenta as informações ao usuário localmente, e os botões dão ao usuário a possibilidade de navegar entre diferentes telas sem depender exclusivamente do ambiente remoto.

4.2 Camada de Processamento

A camada de processamento é executada no ESP32 por meio do firmware em MicroPython. É nessa parte que os valores coletados são interpretados e transformados em informação útil. Além de ler os sensores, o sistema realiza cálculos como conforto térmico, classificação da temperatura, classificação da luz e classificação da qualidade do ar.

Essa camada é importante porque mostra que o dispositivo não apenas transmite números brutos, mas também produz conhecimento a partir deles. Isso é útil para simplificar a visualização no dashboard e para destacar estados do ambiente de forma mais intuitiva.

4.3 Camada de Comunicação

A camada de comunicação conecta o ESP32 ao broker MQTT. Essa conexão é feita via Wi-Fi e utiliza TLS quando configurada com HiveMQ Cloud. O uso de MQTT é adequado porque o protocolo é leve, assíncrono e amplamente utilizado em IoT. A escolha do formato JSON para o payload torna a mensagem flexível e fácil de interpretar em outras ferramentas.

4.4 Camada de Consumo

A camada de consumo é representada pelo Node-RED, que assina o tópico MQTT e transforma os dados em elementos visuais. Essa camada pode exibir valores em medidores, textos e alertas, funcionando como painel de monitoramento em tempo real. Ao ser executado em Docker, o Node-RED ganha facilidade de instalação, isolamento de dependências e reprodutibilidade.

5 Componentes de Hardware

5.1 ESP32

O ESP32 é a unidade central do projeto. Ele foi escolhido por reunir processamento suficiente para aplicações embarcadas, conectividade Wi-Fi integrada e boa compatibilidade com bibliotecas e ferramentas de prototipação. No projeto, ele executa o firmware, lê sensores, processa dados, controla o display e publica mensagens MQTT.

5.2 Sensor DHT22

O DHT22 é responsável por medir temperatura e umidade relativa do ar. Sua escolha se justifica pela simplicidade de uso, ampla documentação e boa aplicabilidade em sistemas de monitoramento ambiental. Ele fornece os dados climáticos fundamentais da estação.

5.3 Sensor LDR

O LDR é um sensor resistivo que varia sua resposta conforme a incidência de luz. No projeto, ele foi usado para estimar a luminosidade do ambiente e permitir a classificação em faixas como baixa, média e alta. Essa informação é útil tanto para leitura local quanto para automação posterior.

5.4 Sensor MQ2

O MQ2 é utilizado para detectar gases combustíveis e fumaça. Em sistemas embarcados, esse tipo de sensor é especialmente útil para monitoramento de segurança. No projeto, sua leitura permite classificar o ar e criar alertas quando o valor ultrapassa determinados limites.

5.5 Display OLED SSD1306

O display OLED SSD1306 foi escolhido por sua interface simples via I2C, baixo consumo e boa visibilidade. Ele permite ao usuário acompanhar os valores mesmo sem depender do dashboard remoto. O uso do display torna o sistema mais autônomo e funcional.

5.6 Botões

Os botões físicos são usados para controlar a navegação entre telas. Isso melhora a interação local com o sistema, permitindo que o usuário alterne entre visão climática, visão de luz/gás e tela de status.

6 Estrutura do Firmware em MicroPython

O firmware foi desenvolvido em MicroPython e organizado em classes para manter o código modular. Essa decisão é importante porque reduz a complexidade do arquivo principal e permite que cada parte do sistema tenha uma função bem definida.

6.1 Configuração do Hardware

A classe `HardwareConfig` centraliza a inicialização de todos os periféricos. Isso inclui o barramento I2C, o display OLED, os sensores analógicos e digitais, além dos botões. Centralizar essa configuração evita repetição e facilita futuras alterações de pinos ou troca de hardware.

```
class HardwareConfig:
    def __init__(self):
        self.i2c = I2C(0, scl=Pin(22), sda=Pin(21))
        self.oled = ssd1306.SSD1306_I2C(128, 64, self.i2c)
        self.dht = dht.DHT22(Pin(4))
        self.ldr = ADC(Pin(34))
        self.ldr.atten(ADC.ATTN_11DB)
        self.gas = ADC(Pin(35))
        self.gas.atten(ADC.ATTN_11DB)
        self.btn1 = Pin(18, Pin.IN, Pin.PULL_UP)
        self.btn2 = Pin(19, Pin.IN, Pin.PULL_UP)
```

6.2 Abstração dos Sensores

A classe `SensorBase` funciona como uma base comum para qualquer sensor do sistema. Essa estrutura facilita a padronização do método `ler()`, permitindo que o coletor trabalhe com diferentes sensores sem precisar conhecer os detalhes internos de cada um.

6.3 Leitura do DHT22

A classe `SensorDHT` encapsula a leitura do DHT22. O método `ler()` faz a medição, captura temperatura e umidade e trata exceções para evitar interrupção do programa principal.

```
class SensorDHT(SensorBase):
    def __init__(self, dht):
        self.sensor = dht

    def ler(self):
        try:
            self.sensor.measure()
            return {
                "temperatura": self.sensor.temperature(),
                "umidade": self.sensor.humidity()
            }
        except:
            return {
                "temperatura": 0,
                "umidade": 0
            }
```

Esse tratamento é importante em sistemas embarcados porque uma falha de leitura não deve derrubar todo o firmware.

6.4 Leituras Analógicas

As classes `SensorLDR` e `SensorMQ2` realizam leituras diretas dos conversores analógico-digitais do ESP32. O uso do ADC é apropriado porque ambos os sensores fornecem um valor contínuo proporcional ao fenômeno físico observado.

6.5 Cálculos e Classificações

A classe `CalculadoraClima` reúne a lógica de interpretação dos dados. Em vez de expor apenas números, o sistema passa a fornecer classificações que fazem mais sentido para o usuário. Isso inclui estado da temperatura, estado do gás, estado da luz, conforto térmico e qualidade do ar.

Essa camada também é útil para padronizar regras. Por exemplo, os limites de temperatura e gás são definidos em um único lugar, o que evita divergências entre ESP32 e Node-RED.

6.6 Coleta e Formatação dos Dados

A classe `DataCollector` percorre todos os sensores, agrupa os resultados em um dicionário e adiciona um timestamp. O payload então é convertido em JSON para publicação MQTT.

```
dados = collector.coletar()
msg = ujson.dumps(dados)
print(msg)
```

Exemplo de payload:

```
{
  "temperatura": 24.5,
  "umidade": 61.0,
  "luminosidade": 1820,
  "gas": 950,
  "timestamp": 1710000000
}
```

6.7 Gerenciamento MQTT

A classe `MQTTManager` abstrai a conexão com o broker, o envio de mensagens e a re-conexão automática. O uso de TLS aumenta a segurança da comunicação.

```
client = MQTTClient(
    CLIENT_ID,
    MQTT_BROKER,
    port=MQTT_PORT,
    user=MQTT_USER,
    password=MQTT_PASSWORD,
    ssl=True,
    ssl_params={"server_hostname": MQTT_BROKER}
)
mqtt = MQTTManager(client, MQTT_TOPIC)
mqtt.conectar()
```

6.8 Controle do Display

A classe `DisplayManager` organiza o conteúdo visual em três telas. Essa divisão evita poluir a interface e ajuda a manter as informações separadas por contexto. O usuário pode alternar entre clima, luz/gás e status do sistema.

7 Fluxo de Dados

O fluxo de dados começa nos sensores físicos e termina na visualização remota. Primeiro, os sensores captam os dados do ambiente. Em seguida, o ESP32 lê esses valores, aplica regras de interpretação e monta um JSON. Depois, a mensagem é enviada ao broker MQTT, onde o Node-RED pode consumi-la e transformar o conteúdo em gráficos, indicadores e alertas.

DHT22, LDR, MQ2, botões e OLED → ESP32 em MicroPython
→ MQTT com TLS → HiveMQ Cloud → Node-RED em
Docker → Dashboard

Figure 1: Fluxo de dados da estação meteorológica IoT

A cadência de envio foi definida para dois segundos, o que oferece boa atualização sem

sobrecarregar a rede. Esse fluxo foi pensado para manter equilíbrio entre tempo de resposta, consumo de recursos e legibilidade da informação.

8 Configuração de MQTT

A comunicação MQTT é um dos pontos centrais do sistema. O ESP32 atua como publicador e o Node-RED como assinante. O broker utilizado é o HiveMQ Cloud, acessado por porta segura 8883. O uso de MQTT é adequado porque o protocolo foi projetado para ambientes com recursos limitados, como microcontroladores e dispositivos IoT.

O payload em JSON permite que diferentes consumidores interpretem os dados sem necessidade de um formato rígido. Além disso, o tópico foi definido de forma simples para facilitar testes e documentação.

9 Node-RED e Docker

O Node-RED foi empregado como ferramenta de supervisão dos dados recebidos. Seu modelo visual facilita a construção de fluxos MQTT, leitura de JSON, processamento de mensagens e criação de dashboards. Ele também permite rápida adaptação de regras e alertas sem alterar o firmware.

A execução em Docker simplifica a instalação e evita conflitos com dependências locais. O contêiner encapsula toda a aplicação e garante maior portabilidade do ambiente.

```
docker run -it -p 1880:1880 -v node_red_data:/data --name mynodered  
nodered/node-red
```

Após a inicialização, o fluxo pode ser importado via arquivo `flow.json`, e o broker MQTT deve ser configurado com os mesmos parâmetros usados pelo ESP32. Isso fecha o ciclo de comunicação entre publicador e consumidor.

10 Simulação em Wokwi

A simulação no Wokwi foi uma etapa importante do desenvolvimento. Ela permitiu testar a montagem dos sensores, a lógica do firmware e a interação com o display antes da implementação física. Essa estratégia reduz erros e acelera o processo de validação.

11 Análise dos Requisitos

11.1 Descrição Detalhada dos Requisitos Funcionais

Além de simplesmente ler sensores, o sistema precisa apresentar um comportamento consistente. Ele deve enviar dados com periodicidade definida, lidar com falhas sem travar, permitir navegação entre telas, manter a comunicação MQTT ativa e organizar os dados em uma estrutura compreensível pelo dashboard.

11.2 Descrição Detalhada dos Requisitos Não Funcionais

A solução precisa ser leve o suficiente para um microcontrolador, clara o bastante para manutenção e flexível para expansão. Também deve manter compatibilidade com o ecossistema MicroPython, utilizar protocolos consolidados e oferecer uma documentação útil para replicação do projeto.

12 Conclusão

O projeto demonstrou a integração entre sensores, firmware embarcado, mensageria MQTT e visualização em dashboard, compondo uma solução IoT completa e funcional. A arquitetura orientada a objetos permitiu manter o código organizado e facilitar a expansão da estação para novas funcionalidades. A combinação de simulação, documentação e implementação prática reforça o caráter didático e técnico da solução.

References

- [1] ESPRESSIF SYSTEMS. *ESP32 Documentation*. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. Acesso em: 03 maio 2026.
- [2] MICROPYTHON. *MicroPython Documentation*. Disponível em: <https://docs.micropython.org/>. Acesso em: 03 maio 2026.
- [3] WOKWI. *Wokwi Documentation*. Disponível em: <https://docs.wokwi.com/>. Acesso em: 03 maio 2026.

- [4] OASIS. *MQTT*. Disponível em: <https://mqtt.org/>. Acesso em: 03 maio 2026.
- [5] HIVEMQ. *HiveMQ Cloud*. Disponível em: <https://www.hivemq.com/mqtt-cloud-broker/>. Acesso em: 03 maio 2026.
- [6] NODE-RED. *Node-RED*. Disponível em: <https://nodered.org/>. Acesso em: 03 maio 2026.
- [7] DOCKER. *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 03 maio 2026.
- [8] FFMPEG. *FFmpeg Documentation*. Disponível em: <https://ffmpeg.org/documentation.html>. Acesso em: 03 maio 2026.
- [9] GITHUB. *Relative links in markup files*. Disponível em: <https://github.blog/news-insights/product-news/relative-links-in-markup-files/>. Acesso em: 03 maio 2026.